

Exploration

```
In [7]: import os
import datetime

import psycpg2
from psycpg2 import sql
from psycpg2.extras import execute_values
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns
```

1.Extract: Chargement des données

1.1 Paramètres + création de la base si nécessaire

```
In [20]: DB_PARAMS = {
    "host": "localhost",
    "port": 65432,
    "user": "postgres",
    "password": "*****",
}
DB_NAME = "customer_personality"
CHEMIN_CSV = "../data/marketing_campaign.csv"

conn = psycpg2.connect(dbname="postgres", **DB_PARAMS)
conn.autocommit = True
with conn.cursor() as cur:
    cur.execute("SELECT 1 FROM pg_database WHERE datname = %s;", (DB_NAME,))
    if cur.fetchone() is None:
        cur.execute(sql.SQL("CREATE DATABASE {};").format(sql.Identifier(DB_NAME)))
        print(f"Base '{DB_NAME}' créée.")
    else:
        print(f"Base '{DB_NAME}' déjà existante.")
conn.close()

CONN_PARAMS = {**DB_PARAMS, "dbname": DB_NAME}
```

Base 'customer_personality' déjà existante.

1.2 création de la table + chargement du CSV (si table vide)

```
In [21]: with psycpg2.connect(**CONN_PARAMS) as conn:
    with conn.cursor() as cur:
        cur.execute("""
            CREATE TABLE IF NOT EXISTS clients (
                id INTEGER PRIMARY KEY, year_birth INTEGER, education VARCHAR(50),
                marital_status VARCHAR(50), income NUMERIC, kidhome INTEGER,
                teenhome INTEGER, dt_customer DATE, recency INTEGER,
                mntwines NUMERIC, mntfruits NUMERIC, mntmeatproducts NUMERIC,
                mntfishproducts NUMERIC, mntsweetproducts NUMERIC, mntgoldprods
                numdealspurchases INTEGER, numwebpurchases INTEGER,
```

```

        numcatalogpurchases INTEGER, numstorepurchases INTEGER,
        numwebvisitsmonth INTEGER, acceptedcmp3 INTEGER, acceptedcmp4 IN
        acceptedcmp5 INTEGER, acceptedcmp1 INTEGER, acceptedcmp2 INTEGER
        complain INTEGER, response INTEGER
    );
    """
    conn.commit()

    cur.execute("SELECT COUNT(*) FROM clients;")
    nb_lignes_en_base = cur.fetchone()[0]

    if nb_lignes_en_base == 0:
        print("Table vide -> chargement du CSV...")
        df_brut = pd.read_csv(CHEMIN_CSV, sep="\t")
        df_brut["Dt_Customer"] = pd.to_datetime(df_brut["Dt_Customer"], form

    colonnes = [
        "ID", "Year_Birth", "Education", "Marital_Status", "Income", "Ki
        "Teenhome", "Dt_Customer", "Recency", "MntWines", "MntFruits",
        "MntMeatProducts", "MntFishProducts", "MntSweetProducts", "MntGo
        "NumDealsPurchases", "NumWebPurchases", "NumCatalogPurchases",
        "NumStorePurchases", "NumWebVisitsMonth", "AcceptedCmp3", "Accep
        "AcceptedCmp5", "AcceptedCmp1", "AcceptedCmp2", "Complain", "Res
    ]
    df_brut = df_brut[colonnes]
    valeurs = [tuple(row) for row in df_brut.to_numpy()]
    execute_values(
        cur,
        f"INSERT INTO clients ({', '.join(df_brut.columns.str.lower())})
        valeurs
    )
    conn.commit()
    print(f"{len(df_brut)} lignes insérées.")
else:
    print(f"Table déjà peuplée ({nb_lignes_en_base} lignes) -> pas de re

```

Table déjà peuplée (2240 lignes) -> pas de rechargement.

1.3 Extraction

```

In [27]: QUERY_EXTRACTION = "SELECT * FROM clients;"

with psycopg2.connect(**CONN_PARAMS) as conn:
    df = pd.read_sql(QUERY_EXTRACTION, conn)

print(f"{len(df)} lignes chargées depuis PostgreSQL.")
df.head()

```

2240 lignes chargées depuis PostgreSQL.

C:\Users\Paul-Emmanuel Buffe\AppData\Local\Temp\ipykernel_23956\1609374055.py:4:
 UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or d
 atabase string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tes
 ted. Please consider using SQLAlchemy.
 df = pd.read_sql(QUERY_EXTRACTION, conn)

Out[27]:

	id	year_birth	education	marital_status	income	kidhome	teenhome	dt_custon
0	5524	1957	Graduation	Single	58138.0	0	0	2012-09-
1	2174	1954	Graduation	Single	46344.0	1	1	2014-03-
2	4141	1965	Graduation	Together	71613.0	0	0	2013-08-
3	6182	1984	Graduation	Together	26646.0	1	0	2014-02-
4	5324	1981	PhD	Married	58293.0	1	0	2014-01-

5 rows × 27 columns



In [28]:

```
# Script de vérification rapide (depuis PostgreSQL)
dates_converties = pd.to_datetime(df['dt_customer'])

print(f"Première inscription enregistrée : {dates_converties.min().strftime('%d-")
print(f"Dernière inscription enregistrée : {dates_converties.max().strftime('%d-
```

Première inscription enregistrée : 30-07-2012

Dernière inscription enregistrée : 29-06-2014

2. Dictionnaire des données (Data Dictionary)

1. People (Profil Client)

- **ID** : Identifiant unique du client.
- **Year_Birth** : Année de naissance du client.
- **Education** : Niveau d'études du client.
- **Marital_Status** : État civil du client.
- **Income** : Revenu annuel brut du foyer du client.
- **Kidhome** : Nombre d'enfants en bas âge dans le foyer.
- **Teenhome** : Nombre d'adolescents dans le foyer.
- **Dt_Customer** : Date d'enregistrement du client auprès de l'entreprise.
- **Recency** : Nombre de jours écoulés depuis le dernier achat.
- **Complain** : 1 si le client s'est plaint au cours des 2 dernières années, 0 sinon.

2. Products (Dépenses par Catégorie)

- **MntWines** : Montant dépensé en vin au cours des 2 dernières années.
- **MntFruits** : Montant dépensé en fruits au cours des 2 dernières années.
- **MntMeatProducts** : Montant dépensé en viande au cours des 2 dernières années.
- **MntFishProducts** : Montant dépensé en poisson au cours des 2 dernières années.
- **MntSweetProducts** : Montant dépensé en confiseries au cours des 2 dernières années.
- **MntGoldProds** : Montant dépensé en produits de luxe/or au cours des 2 dernières années.

3. Promotion (Comportement Campagnes)

- **NumDealsPurchases** : Nombre d'achats effectués avec une réduction.
- **AcceptedCmp1** : 1 si le client a accepté l'offre de la 1ère campagne, 0 sinon.
- **AcceptedCmp2** : 1 si le client a accepté l'offre de la 2ème campagne, 0 sinon.
- **AcceptedCmp3** : 1 si le client a accepté l'offre de la 3ème campagne, 0 sinon.
- **AcceptedCmp4** : 1 si le client a accepté l'offre de la 4ème campagne, 0 sinon.
- **AcceptedCmp5** : 1 si le client a accepté l'offre de la 5ème campagne, 0 sinon.
- **Response** : 1 si le client a accepté l'offre de la dernière campagne marketing, 0 sinon.

4. Place (Canaux d'Achat)

- **NumWebPurchases** : Nombre d'achats effectués via le site Internet de l'entreprise.
- **NumCatalogPurchases** : Nombre d'achats effectués via le catalogue papier.
- **NumStorePurchases** : Nombre d'achats effectués directement en magasin.
- **NumWebVisitsMonth** : Nombre de visites sur le site Internet au cours du dernier mois.

5. Metadata (Données fixes de contrôle)

- **Z_CostContact** : Coût fixe estimé pour contacter un client (valeur constante = 3).
- **Z_Revenue** : Revenu fixe généré par la réponse positive d'un client (valeur constante = 11).

2. Exploration et Analyse des Données (EDA)

```
In [29]: pd.to_datetime(df['dt_customer']).agg(['min', 'max'])
```

```
Out[29]: min    2012-07-30
max    2014-06-29
Name: dt_customer, dtype: datetime64[ns]
```

2.1. Feature Engineering : Calcul de l'Âge et de l'Ancienneté à l'Été 2014

Le Scénario Métier : "Nous nous plaçons dans la posture d'un data analyst au **30 juin 2014**, au lendemain même de la dernière extraction de données. Nous étudions la base client d'une enseigne récente lancée fin juillet 2012, nous offrant ainsi un recul exact de **23 mois** sur son démarrage et sur le comportement d'achat de ses premières cohortes de clients."

Nous définissons cette date du 30/06/2014 comme notre point d'ancrage historique pour recalculer nos variables temporelles :

- **Age** : L'âge du client au moment de l'étude (en 2014).
- **Anciennete_Jours** : Le nombre de jours réels écoulés entre l'inscription du client (`Dt_Customer`) et le 30 juin 2014.

Note : Les colonnes d'origine `Year_Birth` et `Dt_Customer` sont supprimées après traitement.

```
In [30]: DATE_REFERENCE = pd.to_datetime('2014-06-30')

valeurs_age = DATE_REFERENCE.year - df['year_birth']

df['dt_customer'] = pd.to_datetime(df['dt_customer'], format='%d-%m-%Y')
valeurs_anciennete = (DATE_REFERENCE - df['dt_customer']).dt.days

df.drop(columns=['year_birth', 'dt_customer'], inplace=True)

# Insertion de la nouvelle colonne à l'index 1
df.insert(1, 'age', valeurs_age)
df.insert(2, 'ancienne_jours', valeurs_anciennete)
df.head()
```

```
Out[30]:
```

	id	age	ancienne_jours	education	marital_status	income	kidhome	teenhome
0	5524	57	664	Graduation	Single	58138.0	0	0
1	2174	60	114	Graduation	Single	46344.0	1	1
2	4141	49	313	Graduation	Together	71613.0	0	0
3	6182	30	140	Graduation	Together	26646.0	1	0
4	5324	33	162	PhD	Married	58293.0	1	0

5 rows × 27 columns



2.2 Exploration Statistique Globale

```
In [31]: # Résumé statistique des variables de profil et de revenus
df[['age', 'income', 'kidhome', 'teenhome', 'recency']].describe()
```

```
Out[31]:
```

	age	income	kidhome	teenhome	recency
count	2240.000000	2216.000000	2240.000000	2240.000000	2240.000000
mean	45.194196	52247.251354	0.444196	0.506250	49.109375
std	11.984069	25173.076661	0.538398	0.544538	28.962453
min	18.000000	1730.000000	0.000000	0.000000	0.000000
25%	37.000000	35303.000000	0.000000	0.000000	24.000000
50%	44.000000	51381.500000	0.000000	0.000000	49.000000
75%	55.000000	68522.000000	1.000000	1.000000	74.000000
max	121.000000	666666.000000	2.000000	2.000000	99.000000

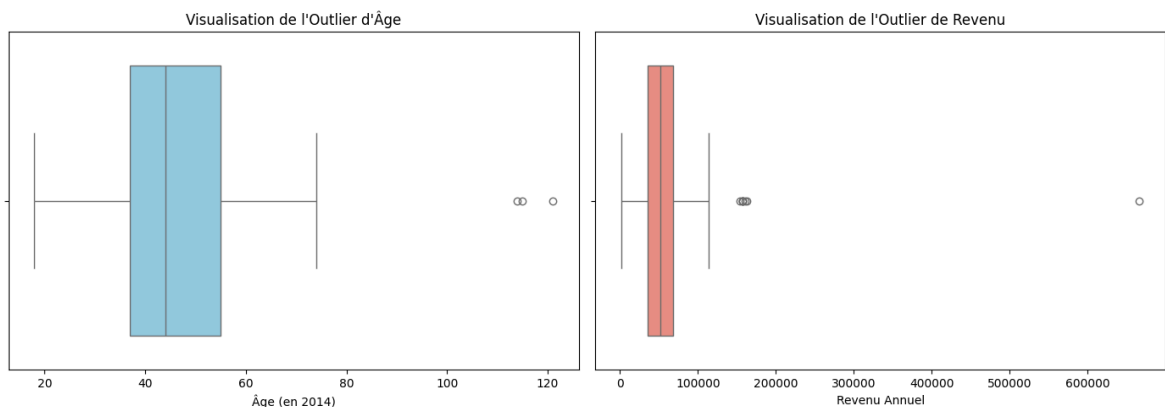
2.3 Exploration Visuelle des Distorsions (Outliers)

```
In [32]: # Configuration de l'affichage (1 ligne, 2 graphiques)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Boxplot de L'Âge (Calculé précédemment sur DATE_REFERENCE)
sns.boxplot(data=df, x='age', ax=axes[0], color='skyblue')
axes[0].set_title("Visualisation de l'Outlier d'Âge")
axes[0].set_xlabel("Âge (en 2014)")

# 2. Boxplot des Revenus (Income)
sns.boxplot(data=df, x='income', ax=axes[1], color='salmon')
axes[1].set_title("Visualisation de l'Outlier de Revenu")
axes[1].set_xlabel("Revenu Annuel")

plt.tight_layout()
plt.show()
```



Synthèse: L'analyse des statistiques globales et des visualisations (Boxplots) sur les 2 240 clients révèle des anomalies nettes et oriente nos choix de nettoyage pour garantir la stabilité du clustering :

- **Âge :** Le minimum de 18 ans valide la conformité réglementaire de la base. En revanche, le graphique isole clairement trois profils aberrants au-delà de 110 ans. Nous appliquons un filtrage strict à 100 ans (`df['Age'] <= 100`) : cela supprime ces anomalies sans perte d'information utile, un choix également cohérent sur le plan métier car ces profils ne sont généralement plus les décideurs d'achat principaux.
- **Revenus (Income) :** La colonne présente 24 valeurs manquantes (qui seront imputées par la médiane) ainsi qu'une erreur de saisie évidente à 666 666 €. Pour la modélisation, nous choisissons un filtrage strict au seuil de 120 000 € (`df['Income'] <= 120000`). Bien que les hauts revenus jusqu'à 200 000 € soient économiquement plausibles, leur écart extrême par rapport à la masse de la clientèle fausserait mathématiquement les distances euclidiennes du K-Means, risquant de monopoliser un cluster entier au détriment de la précision globale.
- **Foyers :** Plus de la moitié des clients n'ont aucun enfant ni adolescent à charge. Les profils avec enfants à charge existent (jusqu'à 4 au total par cumul) mais restent minoritaires.
- **Récence :** La répartition est parfaitement uniforme de 0 à 99 jours. Cet étalement artificiel prouve que les clients inactifs ont été supprimés à la source. Tous les profils étant actifs, cette variable n'aidra pas à séparer les clients lors du clustering.

2.3 Exploration des Variables Catégorielles

```
In [33]: # Analyse des fréquences pour les variables qualitatives majeures
variables_cat = ['education', 'marital_status']

for col in variables_cat:
    print(f"--- Répartition de la variable : {col} ---")
    print(df[col].value_counts(dropna=False))
    print(f"Nombre de modalités uniques : {df[col].nunique()}")
    print("-" * 40 + "\n")
```

--- Répartition de la variable : education ---

education

Graduation 1127

PhD 486

Master 370

2n Cycle 203

Basic 54

Name: count, dtype: int64

Nombre de modalités uniques : 5

--- Répartition de la variable : marital_status ---

marital_status

Married 864

Together 580

Single 480

Divorced 232

Widow 77

Alone 3

Absurd 2

YOLO 2

Name: count, dtype: int64

Nombre de modalités uniques : 8

Synthèse: L'examen des variables qualitatives (**Education** et **Marital_Status**) met en évidence des disparités d'effectifs et des incohérences de saisie. Afin d'optimiser l'Analyse à Facteurs Multiples (AFM) et d'éviter l'instabilité des modèles, les arbitrages suivants ont été retenus :

- **Éducation : * Fusion de 2n Cycle et Master** : Ces deux modalités désignent réglementairement le même niveau universitaire (Bac+5). Leur regroupement crée un bloc statistiquement plus robuste.
 - **Maintien de Graduation (Licence) et PhD (Doctorat) séparés** : Ces diplômes correspondent à des tranches d'âge et des catégories socio-économiques distinctes, indispensables pour qualifier le comportement d'achat.
 - **Maintien de Basic isolé** : Malgré son faible effectif (54 clients), ce profil non-diplômé du supérieur présente un comportement de consommation potentiellement très différent à l'épicerie qu'il convient de ne pas noyer dans la masse.
- **Statut Marital** :

- **Suppression de Absurd** : Cette modalité constitue un bruit informatique pur sans aucune valeur métier. Représentant une part infime du dataset, elle est éliminée.
- **Intégration de Alone et YOLO dans Single** : Alone est un synonyme parfait de célibataire. Pour YOLO, l'isolement de seulement deux individus aurait créé une dimension mathématique vide et instable lors de l'encodage pour l'AFM. Les basculer dans Single permet de conserver leur historique d'achat sans distordre ce grand groupe.
- **Maintien de Widow (Veuf/Veuve) isolé** : Avec 77 clients, cet effectif est suffisant pour être exploité. Fusionner ces profils avec les Single (généralement plus jeunes) détruirait un signal marketing "Senior" très fort, caractérisé par des habitudes d'achat spécifiques (portions, rayons dédiés).

2.4 Cartographie des Valeurs Manquantes

```
In [34]: # Calcul du nombre et du pourcentage de valeurs manquantes par colonne
missing_data = pd.DataFrame({
    'Total Manquants': df.isnull().sum(),
    'Pourcentage (%)': (df.isnull().sum() / len(df)) * 100
})

# On n'affiche que les colonnes qui ont au moins une valeur manquante
missing_data[missing_data['Total Manquants'] > 0]
```

```
Out[34]:
```

	Total Manquants	Pourcentage (%)
income	24	1.071429

Synthèse La colonne des revenus présente 24 valeurs manquantes, soit un taux marginal de 1,07 % sur l'ensemble de la base.

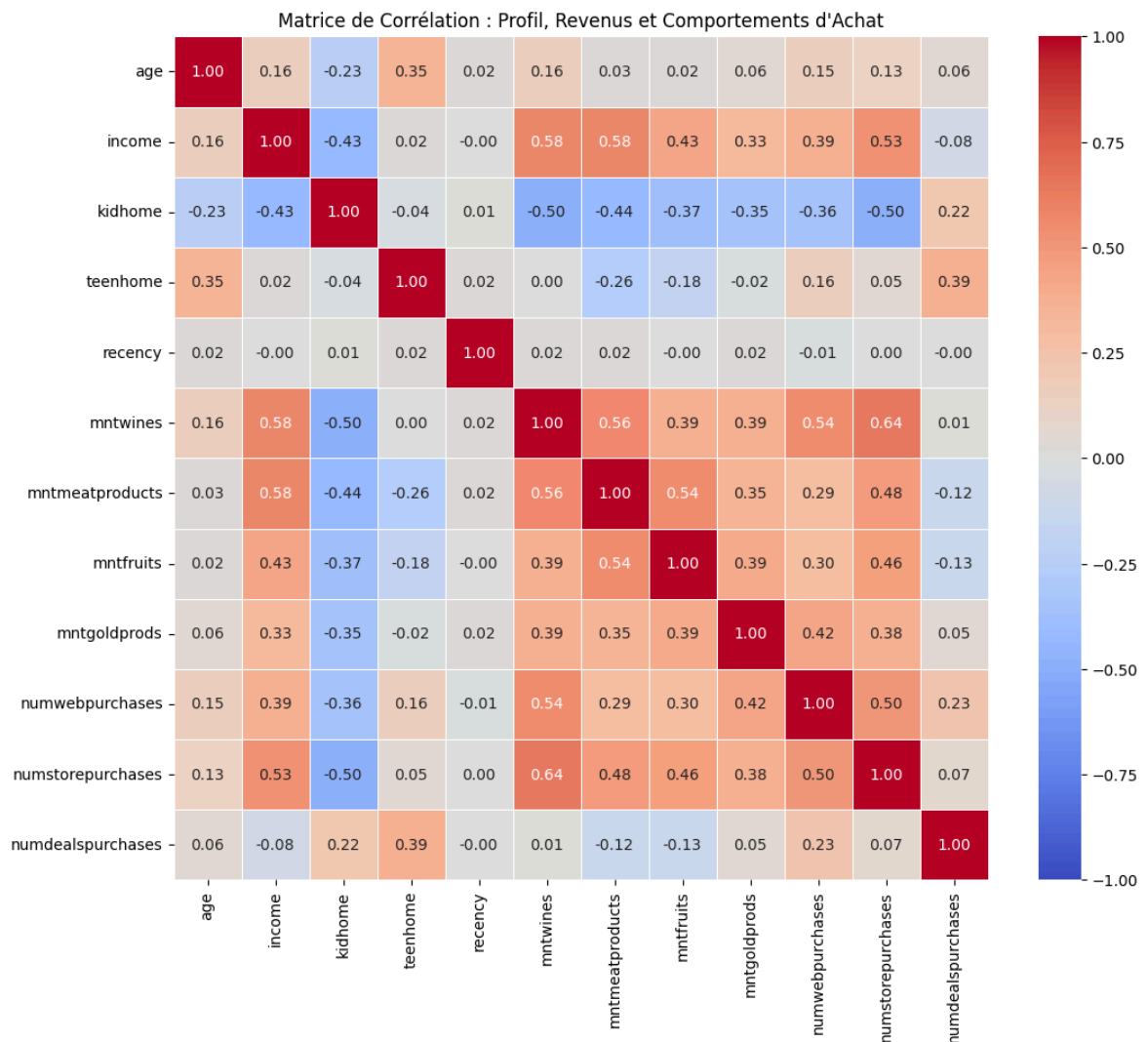
3. Matrice de corrélation

```
In [35]: # 1. Sélection des colonnes numériques (tout en minuscules)
colonnes_analyse = [
    'age',
    'income',
    'kidhome',
    'teenhome',
    'recency',
    'mntwines',
    'mntmeatproducts',
    'mntfruits',
    'mntgoldprods',
    'numwebpurchases',
    'numstorepurchases',
    'numdealspurchases'
]

# 2. Calcul de la matrice de corrélation
matrice_corr = df[colonnes_analyse].corr()
```



```
# 3. Affichage sous forme de Heatmap thermique
plt.figure(figsize=(12, 10))
sns.heatmap(
    matrice_corr,
    annot=True,          # Affiche Les valeurs numériques dans les cases
    fmt=".2f",           # Arrondi à 2 décimales
    cmap="coolwarm",     # Couleur bleu (négatif) à rouge (positif)
    vmin=-1, vmax=1,    # Échelle bloquée entre -1 et 1
    linewidths=0.5
)
plt.title("Matrice de Corrélation : Profil, Revenus et Comportements d'Achat")
plt.show()
```



Synthèse :

- **Validation du biais de Récence** : La variable `Recency` affiche un coefficient de 0.00 avec l'intégralité des autres données (âge, revenus, dépenses). Cette absence totale de corrélation confirme mathématiquement que la récence a été lissée de manière artificielle à la source (exclusion des clients inactifs de plus de 100 jours) et n'apportera aucun pouvoir discriminant à l'algorithme de clustering.
- **Le revenu comme moteur de valeur** : La colonne `Income` présente une forte corrélation positive linéaire avec les rayons premiums comme le vin (0.58) et la viande (0.58). Le pouvoir d'achat s'impose comme le déclencheur principal du

volume d'affaires global et dicte également une plus forte fréquentation des magasins physiques (0.53).

- **L'impact divergent de la structure familiale** : Les enfants en bas âge (Kidhome) et les adolescents (Teenhome) dictent des comportements financiers radicalement opposés. La présence de jeunes enfants fait chuter les budgets plaisir (-0.50 sur le vin et -0.44 sur la viande) au profit d'arbitrages de première nécessité. À l'inverse, la présence d'adolescents n'impacte pas ces rayons mais oriente fortement le foyer vers une stratégie de chasse aux prix (corrélation de 0.39 avec le nombre de promotions utilisées).
- **Identification des blocs sémantiques pour l'AFM** : Les dépenses des différents rayons sont corrélées positivement entre elles (logique de panier global), mais chaque catégorie conserve une signature comportementale unique (ex: le vin est fortement corrélé au magasin physique, l'or aux achats web). Au lieu de les agréger en un score unique et de détruire cette granularité précieuse, ces corrélations valident l'utilisation d'une Analyse à Facteurs Multiples (AFM). Nous conserverons ces variables brutes et les regrouperons dans un bloc sémantique "Dépenses" dédié. L'algorithme pourra ainsi pondérer ce bloc corrélé sans qu'il n'écrase les autres dimensions du profil client.

Signaux faibles :

- **Le rayon Or, un produit « Web-First »** : C'est le seul produit du dataset plus corrélé aux achats en ligne (0.42) qu'en magasin (0.38), révélant une dynamique d'achat d'impulsion ou de cadeaux stimulée spécifiquement par le canal digital.
- **Le Vin et le Magasin, le couple sacré** : La corrélation entre les dépenses de vin et les achats physiques est la plus élevée de toute la matrice (0.64), prouvant que le point de vente physique reste le pilier incontournable et l'épicentre de l'expérience client pour ce produit.
- **Le contre-pied de l'âge et du numérique** : L'âge présente une corrélation légèrement plus forte avec les achats web (0.15) qu'avec le réseau physique (0.13), brisant l'idée reçue d'une clientèle senior déconnectée ou réfractaire aux parcours omnicanaux.

Relations croisées pour le ciblage comportemental :

- **Le Web comme vecteur de promotions** : Le nombre d'achats en promotion est positivement lié aux achats sur internet (0.23) mais indépendant des achats en magasin (0.07). Les comportements opportunistes (chasse aux prix) sont donc digitalisés.
- **L'immunité promotionnelle des hauts revenus** : La corrélation négative entre Income et NumDealsPurchases (-0.08) démontre que la clientèle aisée est insensible aux mécaniques promotionnelles, achetant principalement au prix fort.
- **Le paradoxe budgétaire des adolescents** : La présence d'adolescents fait baisser le budget global alloué à la viande (-0.26). Cela indique une transition d'un panier "gourmet" (pièces nobles) vers un panier "volume/économique" (achats de masse), une distinction cruciale pour segmenter les profils familiaux.

- **La réalité de l'omnicanalité** : La forte corrélation entre les achats web et les achats en magasin (0.50) prouve que les meilleurs clients de l'enseigne naviguent activement entre les deux canaux, validant l'existence d'un profil de consommateur hybride.

```
In [38]: # =====
# REQUÊTE SQL DE TRANSFORMATION (ELT)
# =====
# 1. On calcule La médiane dans une CTE (Common Table Expression)
# 2. On applique Le Feature Engineering (Âge, Ancienneté)
# 3. On nettoie Les catégories (CASE WHEN)
# 4. On filtre Les outliers dans Le WHERE
# 5. On exclut 'recency', 'dt_customer' et 'year_birth' (non sélectionnés)
# Note : Les agrégations manuelles (mnt_total, etc.) ont été retirées
# pour préserver La granularité en vue de L'Analyse à Facteurs Multiples (AFM).

SQL_TRANSFORM = """
DROP TABLE IF EXISTS clients_transformed;
CREATE TABLE clients_transformed AS
WITH median_calc AS (
    SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY income) as med_income
    FROM clients
)
SELECT
    id,
    (2014 - year_birth) AS age,
    (DATE '2014-06-30' - dt_customer) AS anciennete_jours,
    CASE
        WHEN education = '2n Cycle' THEN 'Master'
        ELSE education
    END AS education,
    CASE
        WHEN marital_status IN ('Alone', 'YOLO') THEN 'Single'
        ELSE marital_status
    END AS marital_status,
    COALESCE(income, (SELECT med_income FROM median_calc)) AS income,
    kidhome,
    teenhome,
    mntwines, mntfruits, mntmeatproducts, mntfishproducts, mntsweetproducts,
    numdealspurchases, numwebpurchases, numcatalogpurchases, numstorepurchas
    acceptedcmp3, acceptedcmp4, acceptedcmp5, acceptedcmp1, acceptedcmp2,
    complain, response
FROM clients
WHERE (2014 - year_birth) <= 100
    AND (income <= 120000 OR income IS NULL)
    AND marital_status != 'Absurd';
"""

with psycopg2.connect(**CONN_PARAMS) as conn:
    with conn.cursor() as cur:
        print("Lancement de la transformation en base de données...")
        cur.execute(SQL_TRANSFORM)
        conn.commit()

        cur.execute("SELECT COUNT(*) FROM clients_transformed;")
        nb_lignes = cur.fetchone()[0]
        print(f"Transformation terminée ! La table 'clients_transformed' contient
```

```
df_transformed = pd.read_sql("SELECT * FROM clients_transformed;", conn)

df_transformed.head()
```

Lancement de la transformation en base de données...

Transformation terminée ! La table 'clients_transformed' contient 2203 lignes.

C:\Users\Paul-Emmanuel Buffe\AppData\Local\Temp\ipykernel_23956\3034204217.py:54:
UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
df_transformed = pd.read_sql("SELECT * FROM clients_transformed;", conn)
```

Out[38]:

	id	age	anciennete_jours	education	marital_status	income	kidhome	teenhome
--	----	-----	------------------	-----------	----------------	--------	---------	----------

0	5524	57	664	Graduation	Single	58138.0	0	(
1	2174	60	114	Graduation	Single	46344.0	1	.
2	4141	49	313	Graduation	Together	71613.0	0	(
3	6182	30	140	Graduation	Together	26646.0	1	(
4	5324	33	162	PhD	Married	58293.0	1	(

5 rows × 26 columns



In []: